Week 02 — Lab: What Can a Page Do on Its Own?

100L · Semester 1 · Lab time: 100 min

Deliverable (curriculum §4, week 2): a self-appraisal worksheet: *What can HTML/CSS do alone? Where does JavaScript come in? Why can't HTML handle login?*

You will gather your own evidence first, then use it to answer the three questions:

Part	Time	Format	What you get out of it
0. Pair up	5 min	Pairs	The partner you work with all semester
1. Strip it down	30 min	Pairs	Evidence: what breaks when you take pieces away
2. Who does the job?	20 min	Groups of 4 (two pairs)	Practice placing features in the right layer
3. The worksheet	45 min	Individual	The deliverable

Each pair needs at least one phone. A laptop helps but isn't required. Pairs with no device or no data use the printed screenshot pack instead.

Part 0: Pair up (5 min)

From this week you work in **fixed pairs** for the whole semester. The tutor reads out the pairs.

With your partner, write a short **pair agreement** on one page. Both of you keep a copy:

1. Both your names, and how to reach each other
2. When you'll meet or message during the week to go over the pre-read
3. What you'll do if one of you misses a session
4. One thing each of you is good at that could help the other

---

Part 1: Strip it down (30 min)

Take a working page or app, remove one piece, and write down what still works. Record everything in an **evidence table** like this one:

#	What we did	Site or app	What still worked	What broke, or what message appeared
1	Turned JavaScript off			

Run all three experiments. Write down exactly what you saw, including any error messages, word for word.

Experiment A: JavaScript off (12 min)

1. Switch JavaScript off, as in class notes section 3.

2. Visit **three sites of your own choice**, not the ones from the class notes. Try to pick one that's mostly reading, one where you fill something in (a search box or a form), and one that feels like an app (a game, a map, a chat).
3. On each, try the main thing the site is for. Record what happened.
4. **Switch JavaScript back on.** Check that one of your sites works again.

Experiment B: View the source of a login page (10 min)

1. Pick a site you log in to: a social network, an email service, an online shop, or the student portal.
2. View its login page's source, as in class notes section 4. iPhone users: use your partner's phone, a laptop, or the screenshot pack.
3. Search the source for `password`. Record what you found, and what you **didn't** find.
4. **Don't log in** while doing this. You only need the login page itself.

Experiment C: Airplane mode, again (8 min)

1. Compare your two airplane-mode tables from the pre-read.
2. Pick one app that **still partly worked** offline. Turn airplane mode on and use it again.
3. Record exactly which parts worked and which didn't.
4. **Turn airplane mode off.**

Part 2: Who does the job? (20 min)

Two pairs join up. Your group gets **16 feature cards** and a board with four columns:

`HTML & CSS alone` · `JavaScript in the browser` · `Backend` · `Database`

Place each card in the column that does the feature's main job. If a feature needs the server, ask whether its main job is *deciding* (Backend) or *remembering* (Database).

#	Feature
1	A page with a heading, paragraphs and a photo
2	A link that takes you to another page
3	A menu item that changes colour when you point at it
4	A layout that rearranges itself to fit a phone screen
5	A "Show answer" box that opens and closes
6	A form that won't send until you type an email address with an `@` in it
7	An eye icon that shows or hides the password you're typing
8	A counter showing how many characters you have left as you type
9	A map you can drag and zoom
10	A button that adds up the prices in your shopping list

#	Feature
11	Checking whether the password you typed is correct
12	Sending you a one-time code (OTP) by SMS
13	Showing *your* results, and nobody else's
14	Your cart, still there when you log in on a different phone
15	The number of likes other people gave your post
16	Stopping a student from changing their own grade

The tutor reveals the answers card by card. Your group scores one point for each card in the right column, and you must be able to say why. **At least two cards will surprise you.** Write down which ones, and why you got them wrong.

Part 2 is practice and isn't marked. In Part 3, **use your own Part 1 evidence**, not these cards.

Part 3: The worksheet (45 min): the deliverable

Work alone, on paper. Your partner may not write your answers, and you may not copy theirs.

Section A: Where I am now

Use your pre-read Task 2 answers. Rewrite them here, and improve them now that you know more.

1. **Why I'm here.** Why you chose this programme, and what you want to be able to build in four years.
2. **What I've done so far.** Anything you've tried with computers or code. "Nothing yet" is fine.
3. **What I'm working with.** Your device, and how reliable your data or Wi-Fi is.
4. **Rate yourself today**, from 1 (never heard of it) to 5 (could explain it to a friend):

Topic	1–5
How a page gets from a server to my screen	
What HTML, CSS and JavaScript each do	
The difference between frontend and backend	
What an API is	
Using a computer's files and folders	
Typing quickly and accurately	

5. **What interests me right now:** frontend, backend, mobile, or another job from class notes section 7. Give one reason. (This isn't your track choice; it will probably change.)
6. **My goal for this semester**, and **one thing that could get in the way**, with your plan for it.

Section B: The three questions

Answer each question in **your own words**, in four to eight sentences. Each answer must refer to **at least one observation from your own evidence table**, by its number: what you did, on which site, and what you saw.

1. **What can HTML and CSS do alone?**
2. **Where does JavaScript come in?**
3. **Why can't HTML handle login?** Use the words *frontend*, *backend* and *database* correctly.

Section C: My app, mapped

Choose **one app or website you use often** (not one from the class notes).

1. Draw it as a small diagram: the frontend, the backend, the database, and any API it might call. Label every arrow, as in week 1.
2. Under the diagram, name **one thing that happens on your device** and **one thing that must happen on the server**, and say why.

Optional extras (no extra marks)

- Find a site that still works almost completely without JavaScript, and one that shows nothing at all. What kind of site is each?
- Open the Open-Meteo address from class notes section 5, and change the latitude and longitude to your home town's. (Search the town's name with the word "coordinates".) Does the temperature match how it feels?
- Write your six SDLC sentences from class notes section 8 for a second problem.

Submission

Git is taught in week 11. Until then, submit work by photo:

1. Photograph your worksheet (Sections A, B and C), your evidence table, and your pair agreement.
2. Name the files `week-02-<surname>-<firstname>-<n>.jpg`, numbered 1, 2, 3 and so on.
3. Upload them to the class submission folder the tutor shares in the room, **before the next session**.

Keep the paper. You will look at Section A's ratings again at the end of the semester.

Marking (10 points)

Criterion	Points	What earns full marks
Section A: self-appraisal	2	Every question answered, specifically and honestly. A real goal with a real plan, not "work hard".
Section B, question 1	2	Correct, in the student's own words, backed by their own evidence
Section B, question 2	2	Correct, in the student's own words, backed by their own evidence

Criterion	Points	What earns full marks
Section B, question 3	2	Explains *where* the check must happen and *why*, using frontend, backend and database correctly
Section C: my app, mapped	1	Labelled diagram, with one device-side and one server-side job, each with a reason
Evidence table	1	All three experiments recorded, with what was actually seen

Section A is marked for completeness and honesty, never for how much you already know. A self-rating of 1 scores as well as a 5. The pair agreement and Part 2 aren't marked.

Tutor notes

Kit

Print one set per group of four:

- **Feature cards** 1–16 from the Part 2 table, one per card, large enough to read across a desk.
- **Column board**: an A3 sheet, or four sheets in a row, headed `HTML & CSS alone` · `JavaScript in the browser` · `Backend` · `Database`.

Print one **screenshot pack** for every pair without a working device or data:

- Wikipedia and Excalidraw, each with JavaScript on and off
- Two more sites with JavaScript on and off: one search page and one web app. Label each with the site's name so students can record it.
- The source of a login page, with the `type="password"` line highlighted
- A "before and after" pair for a chat app opened in airplane mode

Pairs using the pack may cite it in their evidence table. Mark this "from the pack" rather than penalising it.

Making the pairs

The curriculum (§12) fixes pairs from week 2 onward and rotates them once per semester.

- Use the warm-up hands-up on device access. **Every pair should include at least one person with regular laptop access**, from week 4 onward if not yet.
- Mix confidence levels: pair someone who has coded before with someone who hasn't.
- With an odd number of students, make one group of three. Don't leave anyone alone.
- Write the list on the board during the break, and keep a copy. You need it for show & tell and for absences.

Running Part 1

- Students on JavaScript-off experiments often pick three sites that all break. Push for variety: "Find me one that *survives*."

- In Experiment B, watch for students about to type a real password. The point is the login *page*, not logging in.
- Walk round at the 25-minute mark and remind everyone to switch JavaScript back on and airplane mode off.

Answer key (Part 2)

#	Column	Why
1	HTML & CSS alone	Content is HTML; how it looks is CSS
2	HTML & CSS alone	Links are plain HTML
3	HTML & CSS alone	CSS can react to pointing (`:hover`)
4	HTML & CSS alone	CSS can change the layout for screen size, which is week 9's topic
5	HTML & CSS alone	HTML has a built-in open/close box (`<details>`). **Surprise card**: the Answer boxes in the class notes use it.
6	HTML & CSS alone	HTML forms can check the format before sending (`type="email"` , `required`). **Surprise card.** Remind them it's a convenience, not security: the user controls their browser.
7	JavaScript in the browser	Changing the box's type on a tap needs JavaScript, but no server
8	JavaScript in the browser	Counting as you type is calculation, done on the device
9	JavaScript in the browser	Dragging and zooming redraw the page constantly. Map *data* comes from a server, so accept "Backend" if the group explains this.
10	JavaScript in the browser	Adding up is calculation; no server needed
11	Backend	Deciding: the backend compares what you typed with the stored record. Nothing secret can be sent to the browser. Accept "Database" if they explain that the record lives there.
12	Backend	Deciding and sending: the server creates the code and sends the SMS. Accept "Database" if they say the code must be remembered to check it later.
13	Backend	Deciding who you are and which rows you may see. The results themselves live in the database, so accept that with a reason.
14	Database	Two devices only share what is stored on the server
15	Database	Other people's actions are stored centrally
16	Backend	A trusted decision, made where the student can't change it. Accept "Database" if they say grades are *stored* there but the *rule* is in the backend.

What to look for in the worksheets

These are prompts for feedback, not model answers. Don't share a model answer for Section B; the deliverable is graded.

What you see	What it means	Prompt
"HTML can't do anything interactive"	They've missed links, forms and open/close boxes	"What happened when you tapped a link with JavaScript off?"

What you see	What it means	Prompt
"JavaScript can check the password"	They think code in the browser is hidden or trusted	"Where would the correct password have to be for the browser to check it? Who can read that?"
"The database checks the password"	They've merged the backend and the database	"The database remembers. Who decides?"
"HTML isn't secure enough"	The right instinct, but no reason	"What did your Experiment B search find, and what didn't it find?"
Answers copied from the class notes	No evidence of their own	"Which row of your evidence table backs this up?"
Every self-rating is 5, or every one is 1	Not yet an honest appraisal	A quiet word, not a mark-down: "Which one would you explain to me right now?"

Using Section A

Section A is a baseline. Read it before week 3 for patterns that affect the room: no laptop access, unreliable data, previous experience. Recruiting Student Leads in week 3 (curriculum §12) is easier with these answers in hand. Keep the worksheets private; they contain personal information.

Sources

- MDN Web Docs: Client-side form validation (card 6, and why browser checks aren't security) · `<details>`: the Details disclosure element (card 5) · Introduction to the server side
- Curriculum document §4 (week 2 deliverable) and §12 (pairs and Student Leads)